

07-API-Specification

API Specification — Leish! v2

1. Conventions
2. Endpoint Index
3. Authentication & Sessions
4. Catalog APIs
5. Booking APIs
6. Messaging APIs
7. Payments Webhook
8. Profile Claim
9. Reviews
10. Data Rights
11. Email Preferences
12. Upload
13. Admin APIs (all behind `requireAdmin`)
14. Health & Cron
15. Error Handling
16. Security Notes (API Consumers)
17. Example: End-to-End Booking Flow via API

API Specification — Leish! v2

FIELD	VALUE
Document ID	LEISH-API-v2.0
Version	2.0.0
Date	2026-08-29
Base URL	<code>https://leish.my</code> (prod) / <code>http://localhost:3000</code> (dev) — <code>NEXT_PUBLIC_SITE_URL</code>
Auth	<code>Cookie: leish_session=<JWT></code> (httpOnly, SameSite=lax); no Bearer header
Content-Type	<code>application/json</code> unless noted (<code>text/event-stream</code> , <code>application/pdf</code> , <code>text/html</code>)
Errors	JSON envelope <code>{error: string}</code> with appropriate HTTP status; zod validation returns first issue message
Helpers	<code>src/server/http.ts</code> — <code>jsonError(message, status)</code> , <code>readJson(req)</code> , <code>tryRoute</code> / <code>statefulRoute</code> wrappers that log <code>pino</code> + normalize errors

1. Conventions

ITEM	RULE
Status codes	200 OK, 201 Created, 400 Bad Request (validation), 401 Unauthorized (missing/bad session), 403 Forbidden (role mismatch), 404 Not Found, 409 Conflict (duplicate/slot), 429 Too Many Requests, 500 Internal
Timestamps	ISO-8601 strings <code>YYYY-MM-DDTHH:mm:ss.sssZ</code> (UTC)
Money	Integer sen (cents). Presented as <code>formatRM(sen) → "RM X.XX"</code> client-side
Pagination	<code>?limit=1..100 default 20, ?offset=0.. default 0</code> → response <code>{pagination: {total, limit, offset, hasMore}}</code> + array
Rate limit	<code>429</code> + <code>Retry-After: <seconds></code> when exceeded; Sliding window via Upstash or in-memory fallback
CSR	<code>ALLOWED_ORIGINS</code> checked for state-changing (<code>POST/PATCH/DELETE</code>) cross-origin; otherwise standard <code>SRS-F-12</code>
Validation	Zod schemas in <code>src/server/validation.ts</code> enforce on every mutating endpoint

2. Endpoint Index

GROUP	METHOD	PATH	AUTH	ROUTE HANDLER
Health	GET	/api/health	No	src/app/api/health/route.ts
Auth — Register	POST	/api/auth/register	No	src/app/api/auth/register/route.ts
Auth — Login	POST	/api/auth/login	No	src/app/api/auth/login/route.ts
Auth — Logout	POST	/api/auth/logout	Cookie	src/app/api/auth/logout/route.ts
Auth — Me	GET	/api/auth/me	Cookie (soft)	src/app/api/auth/me/route.ts or GET /api/me variant
Auth — Verify Email	GET	/api/auth/verify-email?token=	No	src/app/api/auth/verify-email/route.ts
Auth — Resend Verify	POST	/api/auth/resend-verification	Cookie	src/app/api/auth/resend-verification/route.ts
Auth — Forgot	POST	/api/auth/forgot-password	No (rate-limited)	src/app/api/auth/forgot-password/route.ts
Auth — Reset	POST	/api/auth/reset-password	No	src/app/api/auth/reset-password/route.ts
Auth — Errors	POST	/api/errors	No (rate-limited)	src/app/api/errors/route.ts
Catalog — Artists	GET	/api/artists?query&state&area&bridal&nonBridal&budget	No	src/app/api/artists/route.ts
Catalog — Artists (client)	GET	/api/catalog/artists	No	src/app/api/catalog/artists/route.ts
Catalog — Studios	GET	/api/catalog/studios (or /api/studios)	No	src/app/api/catalog/studios/route.ts

GROUP	METHOD	PATH	AUTH	ROUTE HANDLER
Catalog — Artist by slug	GET	/artists/[slug] (RSC, not API)	No	src/app/artists/[slug]/page.tsx (resolveArtist)
Bookings — List	GET	/api/bookings?limit&offset	Cookie	src/app/api/bookings/ route.ts:86
Bookings — Create	POST	/api/bookings	Cookie	src/app/api/bookings/ route.ts:151
Bookings — Transition	PATCH	/api/bookings/[id]	Cookie	src/app/api/bookings/[id]/ route.ts
Bookings — Quotation	POST	/api/bookings/[id]/quotation	Cookie (claimed artist)	src/app/api/bookings/[id]/ quotation/ route.ts
Bookings — Pay Fee	POST	/api/bookings/[id]/pay-fee	Cookie (owner)	src/app/api/bookings/[id]/ pay-fee/ route.ts
Bookings — Pay Balance	POST	/api/bookings/[id]/pay-balance	Cookie (owner)	src/app/api/bookings/[id]/ pay-balance/ route.ts
Bookings — Remind	POST	/api/bookings/[id]/remind	Cookie (claimed artist)	src/app/api/bookings/[id]/ remind/route.ts
Bookings — Refund	POST	/api/bookings/[id]/refund	Cookie (owner)	src/app/api/bookings/[id]/ refund/route.ts
Bookings — Invoice HTML	GET	/api/bookings/[id]/invoice	Cookie (owner artist)	
Bookings — Invoice PDF	GET	/api/bookings/[id]/invoice.pdf	Cookie (owner artist)	
Messages — List	GET	/api/bookings/[id]/messages	Cookie (participant)	src/app/api/bookings/[id]/ messages/ route.ts
Messages — Create	POST	/api/bookings/[id]/messages	Cookie (participant)	src/app/api/bookings/[id]/ messages/ route.ts

GROUP	METHOD	PATH	AUTH	ROUTE HANDLER
Messages — Stream	GET	<code>/api/bookings/[id]/messages/stream</code>	Cookie (participant)	<code>src/app/api/bookings/[id]/messages/stream/route.ts</code> (SSE)
Payments — Webhook	POST	<code>/api/payments/webhook</code>	HMAC (<code>X-Billplz-Signature</code>)	<code>src/app/api/payments/webhook/route.ts</code>
Artists — Claimed	GET	<code>/api/artist-profiles</code>	Cookie	<code>src/app/api/artist-profiles/route.ts</code>
Artists — Claim	POST	<code>/api/artist-profiles</code>	Cookie (artist/studio)	<code>src/app/api/artist-profiles/route.ts</code>
Artists — Edit Self	PATCH	<code>/api/artist-profiles</code>	Cookie (claimed)	<code>src/app/api/artist-profiles/route.ts</code>
Studios — Claim	POST/ GET/ PATCH	<code>/api/studio-profiles</code>	Cookie	<code>src/app/api/studio-profiles/route.ts</code>
Reviews — Add	POST	<code>/api/artists/[slug]/reviews</code>	Cookie	<code>src/app/api/artists/[slug]/reviews/route.ts</code> (or via <code>src/server/catalog.ts</code> helper)
Me — Export	GET	<code>/api/me/export</code>	Cookie	<code>src/app/api/me/export/route.ts</code>
Me — Delete	DELETE	<code>/api/me?confirm=1</code>	Cookie	<code>src/app/api/me/route.ts</code>
Email — Preferences	GET/ PATCH	<code>/api/email/preferences</code>	Cookie	<code>src/app/api/email/preferences/route.ts</code>
Email — Outbox (admin)	GET	<code>/api/admin/emails</code>	Cookie admin	<code>src/app/api/admin/emails/route.ts</code>
Upload	POST	<code>/api/upload</code>	Cookie	<code>src/app/api/upload/route.ts</code>
Cron — Expire	POST/ GET	<code>/api/cron/...</code>	<code>CRON_SECRET</code>	<code>src/app/api/cron/*</code>
Admin — Dashboard	GET	<code>/api/admin</code>	Cookie admin	<code>src/app/api/admin/route.ts</code>

GROUP	METHOD	PATH	AUTH	ROUTE HANDLER
Admin — Users	GET/ POST/ PATCH/ DELETE	<code>/api/admin/users...</code>	admin	<code>src/app/api/admin/users/*</code>
Admin — Artists/ Studios	GET/ POST/ PATCH	<code>/api/admin/artists</code> , <code>/api/admin/studios</code>	admin	<code>src/app/api/admin/artists/*</code>
Admin — Bookings	GET/ PATCH	<code>/api/admin/bookings...</code>	admin	<code>src/app/api/admin/bookings/*</code>
Admin — Payments	GET	<code>/api/admin/payments</code>	admin	<code>src/app/api/admin/payments/route.ts</code>
Admin — Payouts	GET/ PATCH	<code>/api/admin/payouts</code>	admin	<code>src/app/api/admin/payouts/route.ts</code>
Admin — Quotations	GET	<code>/api/admin/quotations</code>	admin	<code>src/app/api/admin/quotations/route.ts</code>
Admin — Messages	GET	<code>/api/admin/messages</code>	admin	<code>src/app/api/admin/messages/route.ts</code>
Admin — Audit	GET	<code>/api/admin/audit</code>	admin	<code>src/app/api/admin/audit/route.ts</code>
Admin — Settings	GET/ PATCH	<code>/api/admin/settings</code>	admin	<code>src/app/api/admin/settings/route.ts</code>
Admin — Analytics	GET	<code>/api/admin/analytics</code>	admin	<code>src/app/api/admin/analytics/route.ts</code>
Dev — Emails	GET	<code>/dev/emails</code> (RSC page)	No (dev-only guard)	<code>src/app/dev/emails/page.tsx</code>
Errors ingestion	POST	<code>/api/errors</code>	rate-limited	<code>src/app/api/errors/route.ts</code>

Routes shown are exhaustive for v2 baseline; exact file layout under `src/app/api/` matches table. See `src/app/api/admin/list-routes.test.ts` for automated coverage of admin surface.

3. Authentication & Sessions

POST /API/AUTH/REGISTER

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Nurul Huda",
  "email": "nurul@example.my",
  "password": "Str0ng!Pass99",
  "role": "customer",          // customer | artist | studio
  "consent": true,
  "consentTimestamp": "2026-08-29T03:00:00.000Z"
}
```

STATUS	BODY
201	{user:{id,email,name,role,emailVerified}, devVerifyUrl?: string} + Set-Cookie: leish_session=...; HttpOnly; Path=/; SameSite=Lax; Secure@prod
400	{error:"Name must be at least 2 characters"} (first zod issue)
409	{error:"An account with this email already exists"}
429	{error:"Too many requests"} + Retry-After

Validation: `registerSchema` `src/server/validation.ts:5`.

POST /API/AUTH/LOGIN

```
POST /api/auth/login
{ "email":"nurul@example.my", "password":"Str0ng!Pass99" }
```

- 200 + cookie on success; 401 generic on failure; 429 when rate-limited. Agnost `login` tracked.

POST /API/AUTH/LOGOUT

- Reads `Cookie: leish_session=...`, revokes JTI (`UPDATE sessions SET revoked=1`), clears cookie. Always 200.

GET /API/AUTH/ME AND GET /API/ME ALIAS

- 200 `{user: PublicUser | null}` — null when unauthenticated (soft check, not 401).

POST /API/AUTH/RESEND-VERIFICATION

- Cookie required; rate-limited per `user_id` (or IP). 200 on success (`{message:"Verification email sent"}`) + email queued; 401 when unauthenticated; 429 when rate-limited.

POST /API/AUTH/FGOT-PASSWORD

```
POST /api/auth/forgot-password
{ "email": "nurul@example.my", "turnstileToken": "" }
```

- Always `200 {message: "If an account exists ..."} (no enumeration)`. Token `password_resets` hashed sha256; 1h expiry. Dev response may include `devResetUrl`.

POST /API/AUTH/RESET-PASSWORD

```
POST /api/auth/reset-password
{ "token": "raw-token-from-email", "newPassword": "NewStr0ng!Pass" }
```

- 200 on success; 400 on invalid/expired/used/reused; single-use (`used_at` set).

4. Catalog APIs

GET /API/ARTISTS

```
Query: ?
query=string(≤100)&state=State&area=string(≤80)&bridal=BridalEvent&nonBridal=NonBridalEvent&budget=int
```

```
GET /api/artists?state=Selangor&area=Cyberjaya&bridal=solemnization&budget=70000&query=aisha
→ 200
{
  "artists": [ Artist, ... ] // Artist {id,slug,name,tagline,bio,image,rating,reviewCount,state,area,
                             // priceFrom,specialties,services,bridal,nonBridal,availability,portfolio,
                             // verified,yearsExperience,reviews[]}
```

- Validates `artistsQuerySchema` `src/server/validation.ts:52`. Implements `listArtists(filters)` with SQL pre-filter then `filterArtists`.
- Caching: public; CDN `Cache-Control: public, s-maxage=300, stale-while-revalidate=60` (set on route).
- Errors: 400 on invalid enum (e.g. unknown `bridal` id).

GET /API/CATALOG/ARTISTS & GET /API/CATALOG/STUDIOS

- Same semantics; separate route wrappers used by `ArtistsBrowser` client components that fetch with `.then()`.

GET /API/ARTISTS/[SLUG]/REVIEWS (OR SERVER HELPER)

- `GET` returns `listEntityReviews("artist", id) → Review[]`.
- `POST` body `{rating:1-5, text: string, event?:string, bookingId?:string}` → `addEntityReview` + `blendAggregate`; see FRS FR-REV-01.

5. Booking APIs

GET /API/BOOKINGS

```
GET /api/bookings?limit=20&offset=0
Cookie: leish_session=...

→ 200
{
  "bookings":[
    {
      "id":"uuid",
      "artistId":"aisha-azman",
      "studioId": null,
      "artistName":"Aisha Azman",
      "service":"Reception Makeup",
      "price":68000, // sen
      "date":"2026-09-10",
      "time":"10:00 AM",
      "notes":"Garden venue",
      "status":"accepted", // requested|accepted|confirmed|cancelled|completed
      "eventType":"reception",
      "venue":"Cyber 5",
      "guestCount":80,
      "quotation": {
        "id":"q-uuid",
        "baseFee":88000,"travelFee":8000,"earlyCallFee":0,"accommodationFee":0,
        "extras":[{"label":"Hair styling","amount":5000}],
        "artistNote":"Early call included",
        "total":101000,
        "status":"pending", // pending|paid|expired|superseded
        "createdAt":"2026-08-29T02:00:00.000Z",
        "expiresAt":"2026-08-30T02:00:00.000Z"
      } | null,
      "totalPrice":101000 | null, // null when no/active quotation expired
      "balanceDueDate":"2026-09-07", // ISO date (event - 3d)
      "balanceAmount":96000, // max(0, total - fee) | null
      "payment": { "amount":5000,"type":"deposit","status":"required|paid|...", "provider":"dev|
billplz","reference":"dev_...","url":null} | null,
      "balancePayment":{"amount":
96000,"type":"balance","status":"...","provider":"...","reference":"...","url":"..."} | null
    }
  ],
  "pagination":{"total":12,"limit":20,"offset":0,"hasMore":false}
}
```

- **Auth:** 401 when missing/bad session.
- **Scoping:** customer → `WHERE user_id=?`; artist/studio → claimed `artist_id / studio_id` set (empty → `[]`).
- **Errors:** 401 unauthenticated.

POST /API/BOOKINGS

```

POST /api/bookings
Cookie: leish_session=...
{
  "artistId": "aisha-azman",
  "service": "Reception Makeup",
  "date": "2026-09-10",
  "time": "10:00 AM",
  "notes": "Garden venue, 30 guests",
  "eventType": "reception",
  "venue": "12-G, Jalan Teknokrat 5",
  "guestCount": 30
}

```

STATUS	BODY
201	{booking:SerializedBooking, user:PublicUser}
400	{error:"Service not available for this artist" zod message}
401	{error:"Not authenticated"}
404	{error:"Artist not found"}
409	{error:"Sorry, this time slot has just been taken..."}
429	Too many requests

See FRS FR-BOOK-01 for server-derived price, pre-check + unique index race guard.

PATCH /API/BOOKINGS/[ID]

```

PATCH /api/bookings/bk_123
Cookie: leish_session=...
{ "action": "accept" } // accept | reject | complete | cancel
→ 200 { booking:SerializedBooking }
→ 400 {error:"Only requested bookings can be accepted"}
→ 401 | 403 | 404

```

Rules per `applyBookingTransition` + ownership checks (FRS FR-BOOK-03). Agnost tracks status updates.

POST /API/BOOKINGS/[ID]/QUOTATION

```
POST /api/bookings/bk_123/quotation
Cookie: leish_session=... (claimed artist)
{
  "baseFee": 88000,
  "travelFee": 8000,
  "earlyCallFee": 0,
  "accommodationFee": 0,
  "extras": [{"label": "Hair styling", "amount": 5000}],
  "artistNote": "Early call included, travel within 30km"
}
→ 201 {quotation: SerializedQuotation}
```

- 403 when not claimed artist; 400 when booking not `accepted` or validation fails; previous pending becomes `superseded`.

POST /API/BOOKINGS/[ID]/PAY-FEE & POST /API/BOOKINGS/[ID]/PAY-BALANCE

```
POST /api/bookings/bk_123/pay-fee
{ "turnstileToken": "..." } // optional when Turnstile configured
→ 200 {payment: {amount: 5000, type: "deposit", status: "required|paid", provider: "dev|
billplz", reference: "...", url: "https://www.billplz.com/bills/xxx" | null},
      booking: {status: "accepted|confirmed"}}
```

- Deposit amount is `getBookingFeeSen()` (not body). Balance amount is `quotation.total - fee`.
- `provider_url` is Billplz hosted page — client opens via `window.open(url, "_blank")`.
- When `provider=dev`, route auto-settles synchronously (feasible for e2e): `markBillPaid` + `handlePaymentPaid` + status `confirmed`.

POST /API/BOOKINGS/[ID]/REMIND

- Claimed artist only, booking must be `confirmed`. Sends `balance_reminder` email respecting `email_preferences.balance_reminder`. Rate-limited.

POST /API/BOOKINGS/[ID]/REFUND

```
POST /api/bookings/bk_123/refund
→ 200 {payment: {...status: "refunded"}, message: "Balance refund issued"}
→ 400 {error: "Only balance payments are refundable" | "Only paid balances..."}
→ 401/403
```

- Refund endpoints call `refundBalancePayment()`; `billplz` path `POST /bills/{id}/refund`.

INVOICE

```
GET /api/bookings/[id]/invoice → 200 text/html (printable)
GET /api/bookings/[id]/invoice.pdf → 200 application/pdf + Content-Disposition: attachment;
filename="leish-invoice-<id>.pdf"
```

- Authz: owner or claimed artist only. Content built server-side from quotation+fees.

6. Messaging APIs

GET /API/BOOKINGS/[ID]/MESSAGES

```
GET /api/bookings/bk_123/messages
→ 200 {messages:[{id, senderName, body, createdAt}]}
```

POST /API/BOOKINGS/[ID]/MESSAGES

```
POST /api/bookings/bk_123/messages
{ "body":"Hello, please confirm venue." }
→ 201 {message:{id, senderName:"", body:"", createdAt:"..."}}
→ 400 empty body
→ 403 not participant
```

GET /API/BOOKINGS/[ID]/MESSAGES/STREAM (SSE)

```
GET /api/bookings/bk_123/messages/stream
Accept: text/event-stream
Cookie: leish_session=...

→ 200 Content-Type: text/event-stream
    Cache-Control: no-cache
    Connection: keep-alive

data: {"id":"m1","senderName":"Nurul","body":"Hi","createdAt":"..."}
data: {"id":"m2","senderName":"Aisha","body":"Hello!","createdAt":"..."}

: keepalive
```

- Replays history `ORDER BY created_at ASC` then streams live via `src/server/chat-bus.ts`. Requires participant auth; drop+reconnect replays.

7. Payments Webhook

POST /API/PAYMENTS/WEBHOOK

```
POST /api/payments/webhook
Content-Type: application/json
X-Billplz-Signature: 64-hex-hmac-sha256-of-raw-body-with-API_KEY
X-Billplz-Paid: true|false (inside body alternative field depending on Billplz version)
{
  "id":"billplz-bill-id",
  "collection_id":"...",
  "paid": true,
  "paid_amount": 5000,
  "amount": 5000,
  "state": "paid"
}
```

STATUS	BODY	EFFECT
200	<code>{ok:true}</code>	Processed: <code>markBillPaid</code> + <code>handlePaymentPaid</code> routed by <code>type</code>
401	<code>{error:"Invalid signature"}</code>	No DB write
404	Bill id not found	No booking confirmed

Verification `verifyBillplzSignature(rawBody, header, apiKey)` `src/server/payments.ts:381`: hex64 regex, `createHmac("sha256",apiKey).update(rawBody).digest("hex")`, `timingSafeEqual`.

Callers must send **raw body bytes** exactly as Billplz did (not pretty-printed).

8. Profile Claim

GET /API/ARTIST-PROFILES

```
GET /api/artist-profiles
Cookie: leish_session=...
→ 200 {profile:{artistId:"aisha-azman", artistName:"Aisha Azman"} | null}
→ 401 unauthenticated
```

POST /API/ARTIST-PROFILES

```
POST /api/artist-profiles
{ "artistId":"aisha-azman" }
→ 201 {profile:{artistId, artistName}}
→ 403 customer role cannot claim
→ 409 artistId already claimed OR user already has profile (PK violation)
→ 404 artist not found
```

PATCH /API/ARTIST-PROFILES

```
PATCH /api/artist-profiles
{ "bio":"New bio...", "priceFrom": 75000, "state":"Selangor" }
→ 200 {artist: Artist}
→ 400 validation on priceFrom range
→ 403 unclaimed cannot edit
```

- Whitelisted fields only; `verified` / `referralEarnings` ignored for self-service.

GET/POST/PATCH /API/STUDIO-PROFILES — MIRRORS ARTIST; SCOPES `STUDIO_ID`.

9. Reviews

POST /API/ARTISTS/[SLUG]/REVIEWS (EXAMPLE; SOME IMPL CALLS **SRC/SERVER/CATALOG.TS** VIA DEDICATED HANDLER)

```
POST /api/artists/aisha-azman/reviews
Cookie: leish_session=...
{
  "rating": 5,
  "text": "Aisha made me look amazing!",
  "event": "Reception",
  "bookingId": "bk_123" // optional; when given gates on that booking
}
→ 201 {review:{id,author,rating,date,text,event}}
→ 400 rating out of 1-5
→ 409 {error:"ALREADY_REVIEWED"} when same booking_id already has review
→ 401
```

10. Data Rights

GET /API/ME/EXPORT

```
GET /api/me/export
Cookie: leish_session=...
→ 200 Content-Type: application/json
  Content-Disposition: attachment; filename="leish-data-a1b2c3d4.json"
  { user, bookings:[], messages:[], payments:[], artistProfiles:[], ...}
→ 401
```

DELETE /API/ME?CONFIRM=1

```
DELETE /api/me?confirm=1
Cookie: leish_session=...
→ 200 {message:"Account deleted"}
→ 400 missing confirm=1
→ 401
```

11. Email Preferences

GET /API/EMAIL/PREFERENCES

→ 200 {preferences:{booking_created, quotation_sent, invoice_sent, quotation_expiry, balance_reminder, status_changed}} all 0/1.

PATCH /API/EMAIL/PREFERENCES

```
PATCH /api/email/preferences
{ "balance_reminder":0 }
→ 200 {preferences:{...}}
```


12. Upload

POST /API/UPLOAD

- Multipart/form-data; image only; routed to Vercel Blob or S3 (`BLOB_READ_WRITE_TOKEN` or AWS). Returns `{url}`. Auth required; size limites configurable.

13. Admin APIs (all behind `requireAdmin`)

All responses are JSON; every mutating handler calls `LogAdminAction()` after success.

GET /API/ADMIN — DASHBOARD AGGREGATES

```
GET /api/admin
→ 200 {
  stats:{
    users:{total, customers, artists, studios, admins},
    bookings:{total, requested, accepted, confirmed, completed, cancelled},
    payments:{total, paid, required, totalRevenue}, // totalRevenue in sen
    artistProfiles: number
  },
  recentBookings:[{id,artist_name,service,date,time,status,created_at}, ...10],
  recentAudit:[{id,action,target_table,target_id,created_at}, ...10]
}
```

GET /API/ADMIN/USERS (+ POST /API/ADMIN/USERS , PATCH /API/ADMIN/USERS/[ID] , DELETE)

- `GET ?limit=&offset=&role=` paginated listing; `POST {name,email,password,role}` create; `PATCH {name,role,emailVerified}` update; `DELETE` remove (audit). Password never returned.

GET /API/ADMIN/ARTISTS (+ POST , PATCH /API/ADMIN/ARTISTS/[ID])

- Mirrors catalog mutability with full whitelist (including `verified` , `referralEarnings`). `POST {name,slug?,tagline,bio,state,area,priceFrom,...}` → `createArtist` .

GET /API/ADMIN/STUDIOS — MIRRORS ARTISTS VIA `CREATESTUDIO` / `UPDATESTUDIO` .

GET /API/ADMIN/BOOKINGS + PATCH /API/ADMIN/BOOKINGS/[ID]

- Admin status override + notes (bypass state machine for ops, but still audited).

GET /API/ADMIN/PAYMENTS — LISTS PAYMENTS JOINED WITH BOOKINGS (FILTER `? STATUS=&TYPE=&BOOKINGID=`).

GET /API/ADMIN/PAYOUTS + PATCH /API/ADMIN/PAYOUTS

```
GET /api/admin/payouts?status=pending
→ 200 {payouts:
  [{id,artist_user_id,booking_id,gross_sen,commission_sen,net_sen,status,settleable_at,settled_at,notes
  ,created_at, artist_name, service, event_date},...]}

PATCH /api/admin/payouts
{ "payoutId":"p_...", "status":"settled", "notes":"DuitNow ref XYZ" }
→ 200 {payout: PayoutRow}
→ 404 payout not found
→ 403 non-admin
```

GET /API/ADMIN/QUOTATIONS — LIST QUOTATIONS WITH BOOKING JOIN, FILTER BY STATUS.

GET /API/ADMIN/MESSAGES — LIST MESSAGES WITH BOOKING SCOPE.

GET /API/ADMIN/EMAILS — RETURNS `EMAIL_OUTBOX` (DEV) / RETRY QUEUE STATUS.

GET /API/ADMIN/AUDIT — PAGINATED `ADMIN_AUDIT_LOG` ORDERED `CREATED_AT DESC`.

GET /API/ADMIN/SETTINGS + **PATCH /API/ADMIN/SETTINGS**

```
GET /api/admin/settings
→ 200 {settings:{booking_fee_sen:"5000", commission_rate_bps:"1000", commission_waiver_sen:"10000"}}

PATCH /api/admin/settings
{ "key":"booking_fee_sen", "value":"7000" } // or batch [{key,value}]
→ 200 {setting:{key,value,updated_by,updated_at}}
→ 400 invalid key/value range
```

GET /API/ADMIN/ANALYTICS — AGNOST ROLLUPS (IF CONFIGURED).

14. Health & Cron

GET /API/HEALTH

```
GET /api/health
→ 200 {ok:true, db:"postgres"|"sqlite", env:"production"|"development", uptime:123.4}
→ 503 if DB unreachable
```

Docker `HEALTHCHECK` hits this.

POST /API/CRON/* (E.G. `/API/CRON/EXPIRE-QUOTATIONS`)

- Guard: `Authorization: Bearer <CRON_SECRET>` (Vercel Cron sends it); otherwise 401.
- Handlers: sweep `findExpiredQuotations()` + `markQuotationExpired`, flush `email_retries` (`next_retry < now && attempts < max`).

15. Error Handling

CONDITION	STATUS	BODY EXAMPLE
Zod validation	400	<code>{error:"Name must be at least 2 characters"}</code>
Missing session	401	<code>{error:"Not authenticated"}</code>
<code>requireAdmin</code> non-admin	403	<code>{error:"Forbidden"}</code>
Unknown artist/booking	404	<code>{error:"Artist not found"}</code>
Duplicate email / profile / slot / payment type	409	<code>{error:"An account with this email already exists"}</code> or slot message
Rate limit	429	<code>{error:"Too many requests"}</code> + <code>Retry-After</code>
Webhook bad sig	401	<code>{error:"Invalid signature"}</code>

Client error ingestion: `POST /api/errors {message, stack?, context}` — rate-limited, `reportError()`.

16. Security Notes (API Consumers)

- Never send `SESSION_SECRET` to client; all `NEXT_PUBLIC_*` are safe.
- Billplz webhook **must** send `X-Billplz-Signature` as hex64 of `HMAC-SHA256(apiKey, rawBody)`; JSON body is the Billplz bill payload (amount in sen).
- All mutating requests should include `Content-Type: application/json` unless multipart; missing `Cookie` returns 401 (not redirect) for API routes; pages handle redirects.
- Rate limits are per-IP + per-user where relevant; clients should respect `429` and back off.

17. Example: End-to-End Booking Flow via API

```
# 1. Register + verify (dev: grab devVerifyUrl from response)
curl -X POST http://localhost:3000/api/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"name":"Nurul","email":"nurul@example.my","password":"Str0ng!
Pass99","role":"customer","consent":true}'

# 2. Create booking
curl -X POST http://localhost:3000/api/bookings \
  -H 'Content-Type: application/json' \
  --cookie "leish_session=..." \
  -d '{"artistId":"aisha-azman","service":"Reception Makeup","date":"2026-09-20","time":"10:00
AM","eventType":"reception","venue":"Cyber 5","guestCount":40}'

# 3. Artist claims + accepts (as artist user)
curl -X POST http://localhost:3000/api/artist-profiles --cookie "leish_session=<artist>" -d '{"artist
Id":"aisha-azman"}'
curl -X PATCH http://localhost:3000/api/bookings/BK_ID --cookie "leish_session=<artist>" -d '{"action
":"accept"}'

# 4. Artist quotes
curl -X POST http://localhost:3000/api/bookings/BK_ID/quotation --cookie "leish_session=<artist>" \
  -d '{"baseFee":88000,"travelFee":8000,"extras":[{"label":"Hair styling","amount":5000}]}'

# 5. Customer pays deposit (dev → auto confirmed)
curl -X POST http://localhost:3000/api/bookings/BK_ID/pay-fee --cookie "leish_session=<customer>" -d
'{}'
# → {payment:{url:null, status:"paid"|"required"}, booking:{status:"confirmed"}}

# 6. Customer pays balance (dev → payout created)
curl -X POST http://localhost:3000/api/bookings/BK_ID/pay-balance --cookie
"leish_session=<customer>" -d '{}

# 7. Artist completes
curl -X PATCH http://localhost:3000/api/bookings/BK_ID --cookie "leish_session=<artist>" -d '{"action
":"complete"}'

# 8. Customer reviews
curl -X POST http://localhost:3000/api/artists/aisha-azman/reviews --cookie "leish_session=<customer>
" \
  -d '{"rating":5,"text":"Amazing!","event":"Reception","bookingId":"BK_ID"}'
```

Next: [docs/08-Traceability-Matrix.md](#) maps each BR → SRS → FRS → API → Table → Test.